

CS3002E Compiler Design

Monsoon 2025

Lecture #13

Bottom-Up Parsing - 2

Department of Computer Science & Engineering
NIT Calicut

Review

- Bottom-up Parsing : Reduction
- Shift-Reduce Parsing

Reduction

- **Reduction**: replaces a substring matching the body of a production by its head (a nonterminal)
 - Given $A \rightarrow \beta$, substring β in a right-sentential form is reduced to A to get the previous right-sentential form.

Handle

- **Handle:** A handle of a right-sentential form γ is a production $A \rightarrow \beta$ and a position in γ where β may be found, such that replacing β at that position by A , produces the previous right-sentential form.
 - For convenience, often the body β is referred to as the handle instead of production $A \rightarrow \beta$.

Handle: Example

$$E \rightarrow E + T \mid T$$

$$T \rightarrow \mathbf{id}$$

Rightmost derivation of $id + id$ (handles underlined)

$$E \Rightarrow \underline{E + T} \Rightarrow E + \underline{id} \Rightarrow \underline{T} + id \Rightarrow \underline{id} + id$$

For the last sentential form $id + id$, the production $T \rightarrow id$, occurring at the first position is a handle - replacing id by T , gives the previous right-sentential form.

Note: A substring α matching the body of $A \rightarrow \alpha$ need not always be a handle.

e.g. the second id (occurring at position following $+$) in the last sentential form is not a handle even though it matches the body of the production $T \rightarrow id$.

Handle Pruning

- Doing a rightmost derivation in reverse, starting with the string of terminals w to be parsed.

If w is a sentence, $w = \gamma_n$, the n^{th} right-sentential form in a derivation of w .

$$S = \gamma_0 \xRightarrow{rm} \gamma_1 \xRightarrow{rm} \dots \xRightarrow{rm} \gamma_{n-1} \xRightarrow{rm} \gamma_n = w$$

- *Handle pruning* starts with reducing a handle $A_n \rightarrow \beta_n$ in γ_n to obtain the previous right-sentential form, γ_{n-1} .
- Reduction steps are repeated with γ_{n-1} and with the sentential forms resulting from successive steps, till the sentential form $\gamma_0 = S$ is obtained or till the parser detects an error.

Shift-Reduce Parsing

Shift-Reduce Parsing is a form of Bottom-Up Parsing

- A stack holds grammar symbols
- An input buffer holds the rest of the input to be parsed
- Initially the stack contains only \$

Shift-Reduce Parsing: Actions

- *Shift*: shift the next input symbol to the top of the stack
- *Reduce*: a set of topmost symbols constitute a handle $A \rightarrow \alpha$. The right end of α is on top of the stack. Pop α and push A .
- *Accept*: stack contains the start symbol and the input is empty
- *Error*: report syntax error or call error recovery routine.

Shift-Reduce Parsing

$S \rightarrow L = R$

$L \rightarrow id$

$R \rightarrow num$

Stack	Input	Action
\$	$id = num\$$	shift
$\$id$	$= num\$$	reduce by $L \rightarrow id$
$\$L$	$= num\$$	shift
$\$L =$	$num\$$	shift
$\$L = num$	\$	reduce by $R \rightarrow num$
$\$L = R$	\$	reduce by $S \rightarrow L = R$
$\$S$	\$	accept

Shift-Reduce Parsing

- Decision on *shift/reduce/reduce by which production* is based on the stack contents and the next input symbol.
 - locating a handle may require examining a set of topmost symbols, sometimes scanning the entire stack.

Parser States

Stack	Input	Action	What the parser has seen
\$	<i>id</i> = <i>num</i> \$	shift	nothing
\$ <i>id</i>	= <i>num</i> \$	reduce by $L \rightarrow id$	substring <i>id</i>
\$ <i>L</i>	= <i>num</i> \$	shift	a string derivable from <i>L</i>
\$ <i>L</i> =	<i>num</i> \$	shift	a string derivable from $L =$
\$ <i>L</i> = <i>num</i>	\$	reduce by $R \rightarrow num$	a string derivable from $L = num$
\$ <i>L</i> = <i>R</i>	\$	reduce by $S \rightarrow L = R$	a string derivable from $L = R$
\$ <i>S</i>	\$	accept	a string derivable from <i>S</i>

- Maintain parser states to keep track of the parsing process.
- A state indicates how much of a production is already seen.
- The parser makes shift/reduce decisions based on the current state and the next input symbol.

Parser States

Stack	Input	Action	Current state: stack contents
\$ 0	$id = num\$$	shift	0: empty stack
\$ 0 id 1	$= num\$$	reduce by $L \rightarrow id$	1: handle id
\$ 0 L 1	$= num\$$	shift	1: L
...	...	...	...

- State symbols pushed to stack along with grammar symbols.
- The current state is on top of the stack.
- The parser makes shift/reduce decisions based on the current state and the next input symbol.

Shift-Reduce Parsing: Parser States

$S \rightarrow L = R$

$L \rightarrow id$

$R \rightarrow num$

Stack	Input	Action
\$ 0	$id = num\$$	shift
\$ 0 id 1	$= num\$$	reduce by $L \rightarrow id$
\$ 0 L 2	$= num\$$	shift
\$ 0 L 2 = 3	$num\$$	shift
\$ 0 L 2 = 3 num 4	\$	reduce by $R \rightarrow num$
\$ 0 L 2 = 3 R 5	\$	reduce by $S \rightarrow L = R$
\$ 0 S 6	\$	accept

State Transitions

Stack	Input	Action
\$ 0	<i>id</i> = num\$	shift
\$ 0 <i>id</i> 1	= num\$	reduce by $L \rightarrow id$
\$ 0 <i>L</i> 2	= num\$	shift
\$ 0 <i>L</i> 2 = 3	num\$	shift
\$ 0 <i>L</i> 2 = 3 num 4	\$	reduce by $R \rightarrow num$
\$ 0 <i>L</i> 2 = 3 <i>R</i> 5	\$	reduce by $S \rightarrow L = R$
\$ 0 <i>S</i> 6	\$	accept

- State 0: after shifting *id*, parser moves to state 1.
- First reduction $L \rightarrow id$: pops state symbol 1 and grammar symbol *id*. Parser enters state 0.
- State 0: after pushing *L*, parser moves to state 2.

Shift-Reduce Parsing: Maintaining Parser States

- Starting from the initial state (with just \$ in the stack) the parser goes through a sequence of state transitions.
- The state symbols are pushed to stack along with the grammar symbols. The topmost symbol indicates the current state of the parser.
- The parser makes *shift/reduce* decisions based on the current state and the next input symbol.
- Reduction involves popping state symbols and grammar symbols
- Parser enters *accept* state after seeing the entire right hand side of a production for the start symbol.

Item

Item: A production with a dot at some position - indicates how much of a production is already seen by the parser.

Example: $S \rightarrow L. = R$

Indicates that the parser has seen a string derivable from L and is expecting to see a string derivable from $= R$

Items indicating Parser state

Item	Parser State
$S \rightarrow .L = R$	Expecting to see a string derivable from $L = R$
$S \rightarrow L. = R$	A string derivable from L is read from the input. Expecting to see a string derivable from $= R$
$S \rightarrow L = .R$	A string derivable from $L =$ is seen. Expecting to see a string derivable from R
$S \rightarrow L = R.$	A string derivable from $L = R$ is seen. Parser can reduce by $S \rightarrow L = R$

Items: Exercise

What does each item indicate?

$L \rightarrow .id$

$L \rightarrow id.$

$R \rightarrow .num$

Parser States indicated by set of items

Stack	Input	Action	Set of Items
\$ 0	<i>id</i> = <i>num</i> \$	shift	$\{S \rightarrow .L = R, L \rightarrow .id\}$
\$ 0 <i>id</i> 1	= <i>num</i> \$	reduce by $L \rightarrow id$	$\{L \rightarrow id.\}$
\$ 0 <i>L</i> 2	= <i>num</i> \$	shift	$\{S \rightarrow L. = R\}$
\$ 0 <i>L</i> 2 = 3	<i>num</i> \$	shift	$\{S \rightarrow L = .R, R \rightarrow .num\}$
\$ 0 <i>L</i> 2 = 3 <i>num</i> 4	\$	reduce by $R \rightarrow num$	$\{R \rightarrow num.\}$
\$ 0 <i>L</i> 2 = 3 <i>R</i> 5	\$	reduce by $S \rightarrow L = R$	$\{S \rightarrow L = R.\}$
\$ 0 <i>S</i> 6	\$	accept	

State: Set of items

A parser state is represented by a **set of items**.

- $A \rightarrow \alpha.\beta$ in a state indicates that the parser has already seen a string derivable from α , and is expecting to see a string derivable from β
- $A \rightarrow \alpha\beta.$ in a state indicates that the parser has seen the entire right hand side of production (may lead to a reduction by $A \rightarrow \alpha\beta$)

Reference:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007.

Further reading:

- ALSU chapter 4, section 4.6